Attention Is All You Need

Vaswani et al. (NeurIPS 2017), arXiv:1706.03762

Paper summary for the CS-781 capstone

The paper at a glance

Citation

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin. *Attention Is All You Need*. NeurIPS 2017. Google Brain and Google Research.

One-sentence summary

Replace recurrence and convolution entirely with attention: the **Transformer**, an encoder–decoder built from multi-head self-attention and feed-forward layers.

Why it matters for our capstone

All four of our Phase-1 architectures are Transformers. A2 is closest to this paper's original encoder–decoder design.

Where it appears in our slides

Main deck: "The four architectures".
Glossary: "Transformer building blocks" and "Architectures".

Roadmap

1. The problem
2. The basic idea: attention only
3. Results
4. Assessment and outlook

Why move beyond recurrent networks?

In 2017 the best sequence models were RNNs (LSTMs, GRUs) or CNNs, often joined by attention.

RNNs are sequential: hidden state h_t needs h_{t-1}. Positions in one example cannot be processed in parallel. This hurts most on long sequences.

CNNs are parallel, but relating two distant positions needs many layers: $O(n)$ for ConvS2S, $O(\log n)$ for ByteNet.

Idea: attention already relates any two positions in one step. What if we use *only* attention?

Easy analogy

An RNN reads a 40-page judgment like a clerk reading aloud, one word at a time, remembering everything in one notebook.

A Transformer spreads all 40 pages on a table. Every sentence can glance at every other sentence at once, and many clerks can work in parallel.

Self-attention vs recurrence vs convolution

n = sequence length, d = representation size, k = kernel size, r = neighbourhood size.

Layer type	Cost per layer	Sequential operations	Max path length
Self-attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$
Restricted self-attention	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

- **Path length**: how many steps a signal travels between two positions. Shorter paths make long-range dependencies easier to learn.
- Self-attention is cheaper than recurrence when $n < d$, which was typical for sentences (e.g. $n \approx 50$, $d = 512$).
- For very long inputs, the n^2 term dominates. This is why our A4 uses sliding-window (restricted) attention.

Scaled dot-product attention

Each position has a **query** q, a **key** k and a **value** v. The output for a query is a weighted sum of the values, weighted by query–key similarity:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Why divide by $\sqrt{d_k}$? If q and k have independent unit-variance entries, $q \cdot k$ has variance d_k. Large dot products push the softmax into regions with tiny gradients. Scaling keeps the variance at 1.

Dot-product attention is fast because it is just matrix multiplication.

Easy worked example ($d_k = 2$)

Sentence: "The court dismissed". Query from "dismissed": $q = [1, 1]$.

	key	$q \cdot k / \sqrt{2}$	weight
The	$[0, 1]$	0.71	0.16
court	$[1, 2]$	2.12	**0.67**
dismissed	$[1, 0]$	0.71	0.16

With values The $[1, 0]$, court $[0, 1]$, dismissed $[1, 1]$, the output is $0.16[1, 0] + 0.67[0, 1] + 0.16[1, 1] =$ $[0.33, 0.84]$. "Dismissed" mostly gathers information from "court", *who* dismissed.

Multi-head attention

Instead of one attention over $d_{\text{model}} = 512$ dimensions, run $h = 8$ **heads** in parallel, each on its own learned projections of size $d_k = d_v = 64$:

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V)$$

$$\text{MultiHead} = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$

Total cost is about that of one full-size head.

Why: one head averages everything into one weighted sum. Several heads can attend to *different* things at once.

Intuition in a judgment

In "The respondent, who was represented by counsel, did not appear before the tribunal":

- one head links "did not appear" to "respondent" (subject);
- another links "appear" to "tribunal" (location);
- another links "represented" to "counsel".

The paper's appendix shows heads that track syntax and coreference.

Three uses of attention in the model

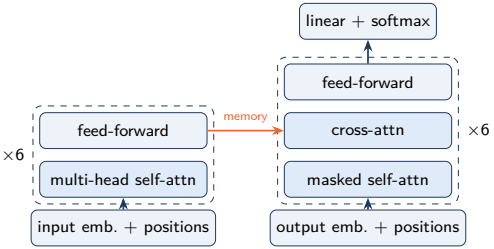
1. **Encoder self-attention**: every source position attends to every source position.
2. **Decoder masked self-attention**: each target position attends only to *earlier* target positions. Future scores are set to $-\infty$ before the softmax, so generation stays auto-regressive.
3. **Encoder–decoder (cross) attention**: decoder queries attend to the encoder's outputs, the keys and values. Each output word can look at the whole input.

Translation example

Translating "The appeal is dismissed" into Hindi:

- encoder self-attention works out that "dismissed" refers to "appeal";
- while writing each Hindi word, the decoder's masked self-attention sees the Hindi words already written;
- cross-attention looks back at the English words it needs next.

The full architecture



- Each sub-layer is wrapped as $\text{LayerNorm}(x + \text{Sublayer}(x))$: a residual connection, then layer normalisation (*post-LN*).
- Feed-forward network, applied to each position separately:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

with $d_{ff} = 2048$ (4× wider).

- Input and output embeddings and the pre-softmax layer share one weight matrix; embeddings are multiplied by $\sqrt{d_{\text{model}}}$.
- Base model: $N = 6$, $d_{\text{model}} = 512$, $h = 8$: 65M parameters.

Positional encoding

Attention ignores order, so position information is *added* to the embeddings:

$$PE_{(pos,2i)} = \sin\left(pos/10000^{2i/d}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(pos/10000^{2i/d}\right)$$

- Wavelengths form a geometric series from 2π to $10000 \cdot 2\pi$.
- For any fixed offset k, PE_{pos+k} is a *linear function* of PE_{pos}. The authors hoped this would let the model attend by relative position.
- Learned position embeddings gave nearly identical results. Sinusoids were chosen because they might extrapolate to longer sequences.

Easy analogy: a clock

Each pair of dimensions is a clock hand turning at its own speed. Fast hands tell neighbouring positions apart; slow hands tell distant ones apart. Together they give each position a unique reading.

In our models

A2 uses these sinusoids. A1 uses learned positions. A3 (ALiBi) and A4 (RoPE) use later relative schemes.

Training recipe

- **Data**: WMT 2014 EN–DE (4.5M sentence pairs, shared BPE vocabulary of about 37k); EN–FR (36M sentences, 32k word-pieces). Batches of about 25k source and 25k target tokens.
- **Hardware**: 8 NVIDIA P100 GPUs. Base: 100k steps in 12 hours. Big: 300k steps in 3.5 days.
- **Adam** with $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$.
- **Learning rate** with warm-up, then inverse-square-root decay:

$$\text{lr} = d_{\text{model}}^{-0.5} \min\left(\text{step}^{-0.5}, \text{step} \cdot \text{warmup}^{-1.5}\right)$$

with $\text{warmup} = 4000$.

Regularisation

- Dropout 0.1 on each sub-layer output and on embedding sums.
- **Label smoothing** $\epsilon = 0.1$: the target puts 0.9 on the true token and spreads 0.1 over the rest. This hurts perplexity but improves BLEU.

Inference

Beam search (beam 4, length penalty 0.6). Averages of the last 5 (base) or 20 (big) checkpoints.

Machine translation (newstest2014)

Model	BLEU		Training FLOPs	
	EN-DE	EN-FR	EN-DE	EN-FR
GNMT + RL	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
ConvS2S ensemble	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

Reading the table

- EN–DE: 28.4 BLEU, **more than 2 BLEU** above the best earlier models, including ensembles.
- EN–FR: a new single-model record of 41.8.
- The base model already beats all earlier EN–DE systems with a fraction of the training compute.

The paper's text states 41.0 for EN–FR in Section 6.1; the abstract and Table 2 give 41.8.

Model variations (EN–DE dev set, newstest2013)

Change from base	PPL	BLEU	Params
Base ($h = 8$, $d_k = 64$)	4.92	25.8	65M
$h = 1$, $d_k = 512$	5.29	24.9	
$h = 4$, $d_k = 128$	5.00	25.5	
$h = 16$, $d_k = 32$	4.91	25.8	
$h = 32$, $d_k = 16$	5.01	25.4	
$d_k = 16$ (smaller keys)	5.16	25.1	58M
$N = 2$ layers	6.11	23.7	36M
dropout 0.0	5.77	24.6	
learned positions	4.92	25.7	
Big ($d = 1024$, $h = 16$)	**4.33**	**26.4**	213M

Lessons

- One head is 0.9 BLEU worse; too many heads also hurt.
- Smaller keys hurt: judging compatibility is not trivial.
- Bigger models are better.
- Dropout matters a lot for avoiding overfitting.
- Learned and sinusoidal positions are about equal.

Perplexities are per word-piece.

Beyond translation: constituency parsing

Parsing turns a sentence into a syntax tree, written as a longer bracketed sequence. It has strong structural constraints, and RNN sequence-to-sequence models struggled with small data.

A 4-layer Transformer ($d_{\text{model}} = 1024$), with almost no task-specific tuning, on WSJ Section 23:

Setting	F1
WSJ only (40K sentences)	91.3
Semi-supervised (about 17M sentences)	92.7

Why it matters

With only the 40K WSJ sentences, the Transformer beat the Berkeley parser (90.4), where RNN sequence-to-sequence models had failed. Only one model in the comparison, the RNN Grammar, did better.

This was early evidence that the architecture is general-purpose, as our seven phases assume.

How our Phase-1 models descend from this paper

Paper component	Our A1–A4	Change and reason
Encoder–decoder	A2 keeps it; A1, A4 decoder-only; A3 one shared stack	decoder-only is simpler for language modelling
Post-LN	pre-LN (A1–A3), RMSNorm (A4)	pre-LN trains more stably when deep
ReLU FFN	ReLU (A2), GELU (A1, A3), SwiGLU (A4)	gated units work better (Shazeer, 2020)
Sinusoidal positions	sinusoids (A2), learned (A1), ALiBi (A3), RoPE (A4)	relative schemes extrapolate better
Multi-head attention	MHA; grouped-query in A4	smaller KV-cache
Warm-up + inverse sqrt	warm-up + cosine	common modern choice
Tied embeddings	tied embeddings	fewer parameters

Strengths and limitations

Strengths

- Fully parallel training over positions.
- Constant path length between any two tokens.
- Better quality at a fraction of the training cost.
- Simple, uniform building blocks; scales with size.
- Attention weights are partly interpretable.

Limitations

- $O(n^2)$ time and memory in sequence length. A problem for long legal judgments.
- Generation is still one token at a time.
- Positions only via added encodings; extrapolation to longer inputs was not tested.
- Post-LN can be unstable in deep stacks without warm-up.
- Evaluated mainly on translation and one parsing task.

Future research directions (1): stated in the paper, and what followed

Direction	What followed
Other modalities *(paper)*	Vision Transformer (Dosovitskiy et al., 2021) for images; Transformers for speech and video
Local, restricted attention for large inputs *(paper)*	Longformer (Beltagy et al., 2020) and sliding windows as in Mistral; FlashAttention (Dao et al., 2022) makes exact attention memory-efficient
Less sequential generation *(paper)*	Non-autoregressive translation; speculative decoding
Pre-training at scale	BERT (encoder-only), GPT (decoder-only), T5 (encoder–decoder)
Better positions	Relative positions (Shaw et al., 2018), RoPE (2021), ALiBi (2022)
Normalisation and FFN	Pre-LN (Xiong et al., 2020); gated FFNs such as SwiGLU (Shazeer, 2020)

Future research directions (2): open questions

Open in general

- Sub-quadratic sequence models that match attention quality, such as state-space models.
- Why attention heads specialise, and which heads matter.
- Scaling laws: the best balance of model size, data and compute.
- Length generalisation beyond training context.

Open for legal NLP

- Reading whole judgments (tens of thousands of tokens) within a free GPU's memory.
- Does the encoder–decoder split (A2) help summarisation more than decoder-only models?
- Can attention maps serve as explanations a lawyer would accept?

Summary

- The Transformer removes recurrence and convolution. It uses **scaled dot-product attention**, **multi-head** attention, position-wise feed-forward layers, residual connections and LayerNorm.
- Masked decoder self-attention keeps generation auto-regressive; cross-attention links decoder and encoder.
- Sinusoidal positional encodings supply word order.
- Results: 28.4 BLEU (EN–DE) and 41.8 (EN–FR), state of the art at much lower training cost. It also generalises to parsing.
- Every model in our capstone is a descendant of this architecture.

Reference

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention Is All You Need. In *Advances in Neural Information Processing Systems 30*. arXiv:1706.03762.

Later work cited: Shaw et al. (2018); Beltagy et al. (2020) Longformer; Xiong et al. (2020) On Layer Normalization in the Transformer Architecture; Shazeer (2020) GLU Variants; Dosovitskiy et al. (2021) ViT; Dao et al. (2022) FlashAttention.